

Applying Architecture-based Adaptation to Automate the Management of Internet-of-Things

Danny Weyns^{1,2}, M. Usman Iftikhar², Danny Hughes^{1,3}, and Nelson Matthys³

¹ imec-DistriNet, Department of Computer Science
Katholieke Universiteit Leuven, B-3001 Heverlee, Belgium
danny.weyns@kuleuven.be

² Linnaeus University, 351 95 Vaxjo, Sweden

³ VersaSense, B-3001 Heverlee, Belgium
info@versasense.com; <https://www.versasense.com/>

Abstract. Architecture-based adaptation equips a software-intensive system with a feedback loop that enables the system to adapt itself at runtime to changes to maintain its required quality goals. To guarantee the required goals, existing adaptation approaches apply exhaustive verification techniques at runtime. However these approaches are restricted to small-scale settings, which often limits their applicability in practice. To tackle this problem, we introduce an innovative architecture-based adaptation approach to solve a concrete practical problem of VersaSense: automating the management of Internet-of-Things (IoT). The approach, called MARTAS, equips a software system with a feedback loop that employs Models At Run Time and Statistical techniques to reason about the system and adapt it to ensure the required goals. We apply MARTAS to a building security case system, which is a representative IoT system deployed by VersaSense. The application comprises a set of IoT devices that communicate sensor data over a time synchronized smart mesh network to a central monitoring facility. We demonstrate how MARTAS outperforms a conservative approach that is typically applied in practice and a state-of-the-art adaptation approach for different quality goals, and we report lessons learned from this industrial case.

Keywords: Architecture-based adaptation · Self-adaptation · Feedback loop · Internet-of-Things · Automated management.

1 Introduction

Handling change is an increasingly important challenge for software engineers. Change can manifest itself in different forms, ranging from market-driven evolution of products to uncertainties a system faces during operation (e.g., sudden changes in the environment or dynamics in the availability of resources). Our particular focus in this paper is on the ability of software systems to handle changes at runtime autonomously. A prominent approach to deal with runtime change is so called architecture-based adaptation [21, 14, 11, 16, 29]. Architecture-based adaptation equips a software system with an external feedback loop that collects data of the system and its environment that was difficult or impossible to determine before deployment. The feedback loop uses the collected data to reason about itself and to adapt itself to changes in order to provide the required quality goals, or gracefully degrade if needed. A typical example is

a self-managing Web-based client-server system that continuously tracks and analyzes changes in load and available bandwidth and dynamically adapts the server configuration to provide the required quality of service to its users, while minimizing costs [11].

Over the past two decades, researchers and engineers have put extensive efforts in understanding how to engineer self-adaptive systems [8, 23, 6, 18, 19]. In recent years, the particular focus of research has been on how to provide assurances for the quality goals of self-adaptive systems that operate under uncertainty [4, 19]. According to [27], after a relatively slow start, research in the field of self-adaptation has taken up significantly from 2006 onwards. The field is now – according to the Redwine and Riddle model [22] – following the regular path of maturation and is currently in the phases of internal and external enhancement and exploration. Self-adaptation techniques have found their way to industrial applications, a prominent example is cloud elasticity [9, 10]. However, there is a broad agreement – at least in the research community – that self-adaptation can contribute to solving many practical problems that originate from the continuous change that systems are exposed to during operation.

Architecture-based adaptation is one prominent approach to realise self-adaptation. In this approach, the external feedback loop is realised by different interacting components that share runtime models. These models include an architectural model of the managed system that allows the feedback loop to reason about different system configurations and adapt the system to realise the adaptation goals. Other prominent complementary approaches to realise self-adaptation are self-aware computing [15], self-organisation [7], and control-based software adaptation [24]. The focus of state-of-the-art approaches for architecture-based adaptation that aim at providing assurances for the quality goals are primarily based on applying formal techniques [4, 30, 19]. Some approaches employ formal methods to provide guarantees by construction. More recently, the use of probabilistic models to handle uncertainties at runtime has particular gained interest. These models are used during operation to verify properties using model checking techniques to support decision-making for adapting the system. However, these approaches use exhaustive verification, which is very demanding in the resources and time required to make adaptation decisions. Hence, these approaches are restricted to small-scale settings, which often limits their applicability in practice. This is particularly the case for resource constrained systems and large-scale IoT setups.

To tackle the limitations of current formal approaches for architecture-based adaptation, we propose MARTAS, a novel approach that combines Models At RunTime And Statistical techniques to realise adaptation. We validate MARTAS for a building security monitoring system, which is a representative case of IoT applications developed by VersaSense, a provider of industrial IoT solutions. The test results show that MARTAS outperforms a conservative approach that is typically used in practice as well as a state-of-the-art adaptation approach that uses exhaustive verification.

The remainder of this paper is structured as follows. In Section 2, we explain the basic principles of architecture-based adaptation. Section 3 introduces the IoT application that we use for the evaluation of this research. In Section 4, we present MARTAS, the novel approach for architecture-based adaptation that combines formal models with statistical techniques at runtime to make adaptation decisions. In Section 5, we evaluate MARTAS for a real-world IoT system deployment. Section 6 discusses related work. In Section 7, we highlight lessons learned, and we draw conclusions in Section 8.

2 Architecture-Based Adaptation in a Nutshell

Self-adaptation and architecture-based adaptation in particular can be considered from two angles: (1) the ability of a system to adapt in response to changes in the environment and the system itself [6]; the “self” prefix indicates that the system adapts with minimal or no human involvement [2], and (2) the feedback loop mechanisms that are used to realize self-adaptation that explicitly separate the part of a system that deals with the domain concerns (goals for which the system is built) from the part that deals the adaptation concerns (the way the system realizes its goals under changing conditions)

In architecture-based adaptation, the feedback loop system maintains an architectural model of the managed system, possibly extended with relevant aspects of the environment in which the system operates. This model is kept up to date at runtime and used to make adaptation decisions, in order to achieve the goals of the system. Using a model at an architectural level provides the required level of abstraction and generality to deal with the self-adaptation problem [16, 28].

An architecture-based adaptive system comprises two key building blocks: a *managed system* and a *managing system*. The managed system comprises the application code that realizes the system’s domain functionality. The managing system manages the managed system; that is, the managing system comprises the adaptation logic that deals with one or more adaptation goals. The adaptation goals represent concerns about how the managed system realises the domain functionality; they usually relate to software qualities of the managed system. Adaptation goals can be subject of change themselves.

A typical approach to structure the software of the managing system is by means of a so-called Monitor-Analyzer-Planner-Executor + Knowledge feedback loop (MAPE-K in short). The Knowledge that is shared among the MAPE components contains various types of runtime models, including models of representative parts of the managed system and the environment, models of the qualities that are subject to adaptation, and other working models that are shared among the MAPE components. The *Monitor* collects runtime data from the managed system and the environment and uses this to update the content of the Knowledge, resolving uncertainties (e.g., the interference of the links in an IoT network is tracked to update the relevant runtime models). Based on the current knowledge, the *Analyzer* determines whether there is a need for adaptation of the managed system using the adaptation goals. If adaptation is required, the *Planner* puts together a plan that consists of a set of adaptation actions that are then enacted by the *Executor* that adapts the managed system as needed.

An example state-of-the-art approach that provides guarantees at runtime for the adaptation goals is QoS MOS (Quality of Service Management and Optimisation of Service-based systems) [3]). QoS MOS models a service-based application as a Discrete Time Markov Chain. The feedback loop employs this formal model to identify the service configurations that satisfy the Quality of Service (QoS) goals using a *model checker*. The result of the analysis is a ranking of the configurations based on the required QoS requirements. The best option is used to reconfigure the service-based system to guarantee the adaptation goals. QoS MOS is a representative state-of-the-art approach that uses exhaustive verification to make adaptation decisions. Due to the state explosion problem (the number of states of the models grows exponentially with the model size, hence also the resources and time required for verification), such ap-

proaches are restricted to small-scale settings, which often limits their applicability in practice. MARTAS aims at contributing to tackle this challenging problem.

3 IoT Application

In this section, we describe a building security monitoring application that we use throughout the remainder of the paper for illustration and to evaluate MARTAS. We start with introducing the problem context. Then, we give an overview of the application setting. The section concludes with explaining the challenge VersaSense faces with the management of this kind of IoT applications.

3.1 Problem Context

VersaSense is a provider of wireless IoT products for industrial sensing and control systems. The company employs an in-house developed IoT technology known as Micro Plug and Play (MicroPnP) [20], which includes a suite of ultra-low power wireless IoT devices (*motest*), a wide range of sensors and actuators, and management either as a cloud integration service or as a dedicated appliance. The company uses an 802.15.4e wireless mesh network (SmartMesh IPTM) for short range factory scenarios and a Long Range low power star network (LoRa⁴) for applications that require multi kilometer range. VersaSense provides IoT solutions directly to industry in the areas of surveillance, facility management, manufacturing, consumer goods, and precision agriculture.

The solutions developed by VersaSense support substantial automation, including automatic identification of sensing and actuation peripherals, installation of their drivers, and static network configuration. However, the management of deployed IoT applications, and in particular handling interference in network connections and sudden changes in traffic load remains a challenge. This is a major impediment to ensuring high quality of service (e.g., multi-year battery lifetimes and high levels of reliability).

The typical approach to tackle this challenge is either by over-provisioning (e.g., power settings are set to maximum and duplicate packets are sent to parents) and/or by hand-tuning the network settings. While this conservative approach may achieve good reliability (low packet loss), it is suboptimal in energy consumption. Furthermore, this approach requires manual interventions for network maintenance that are costly and error prone. In a joint R & D effort between VersaSense and imec-DistriNet we studied and developed an innovative self-adaptation solution to tackle this challenging problem.

3.2 IoT Application

To build a solution that automates the management of IoT applications in an efficient way, VersaSense deployed an IoT application at the Computer Science Campus of KU Leuven using their state-of-the-art technology. This application is a representative case for a medium-scale IoT facility/factory developed by VersaSense. Figure 1 shows a schematic overview of the application that is set up as a smart mesh network with 15 motes that are equipped with different types of sensors. The network uses time synchronized communication with communication slots fairly divided among the motes.

⁴ <https://www.lora-alliance.org/>; <https://www.semtech.com/technology/lora>



Fig. 1. IoT system deployed by VersaSense at the Computer Science Campus in Leuven

Motes are strategically placed to provide access control to labs (RFID sensors), to monitor the movements and occupancy status (passive infrared sensors) and to sense the temperature (heat sensors). The sensor data is relayed from the motes to the IoT gateway that is deployed at a central monitoring facility. Communication in the network is organized in cycles, each cycle comprising a fixed number of communication slots. Each slot defines a sender and receiver mote that can communicate.

The domain concern for the IoT network is to relay sensor data to the gateway. The adaptation concerns are to ensure reliable and energy-efficient communication. The VersaSense stakeholders defined the adaptation goals as follows: (1) the average packet loss over a given time period should not exceed a required threshold, (2) the average latency of packets should be below a given fraction of the cycle time, (3) the energy consumed by the motes should be minimized to optimize the lifetime of the network.

Achieving these goals is challenging due to two primary types of uncertainty: (1) network interference and noise due to factors such as weather conditions and the presence of other WiFi signals in the neighborhood; interference affects the quality of communication which may lead to packet loss; (2) fluctuating traffic load that may be difficult to predict; e.g., packets produced by a passive infrared sensor are based on the detection of motion of humans, which may be difficult to anticipate.

Two factors determine the critical qualities: the power settings of the motes used for communication (from 0 for min power to 15 for max power) and the distribution of the packets sent by each mote over the links to its parents (e.g., for two parents: 0% to one parent and 100% to the other, 20/80, . . . 100/0). Operators can set the power settings and the distribution of packets of the motes in the network via an interface.

The user interface also offers access to sensor data based on user defined properties. These include the traffic load generated by motes, the energy consumed by the motes, the Signal-to-Noise (SNR) ratio of the communication links (SNR represents the ratio between the level of the signal used for communication and the level of the noise from the environment. Lower SNR implies higher interference, resulting in higher packet loss.), and statistical data about the QoS of the overall network for a given period.

3.3 Challenge

The general challenge we aim to tackle in this joint R & D effort is the following:

How to automate the maintenance of smart mess IoT networks to ensure the required quality goals in the face of uncertain operating conditions?

For the evaluation of MARTAS, we defined the following concrete quality requirements that need to be realised regardless of possible network interference and fluctuating load of packets generated in the network:

- R1:** The average packet loss over 24 hours should not exceed 10%;
- R2:** The average packet latency over 24 hours should not exceed 5% of the cycle time;
- R3:** The energy consumed by the motes should be minimized.

When architecture-based adaptation is applied, these quality requirements become the adaptation goals.

4 MARTAS: Novel Approach to Architecture-based Adaptation

We now present MARTAS, the novel approach to architecture-based adaptation that combines formal models with statistical techniques at runtime to make adaptation decisions. We start with a general overview of MARTAS. Then we show how we instantiated the approach to realise the adaptive IoT application.

4.1 Decision Making with Formal Models and Statistical Techniques

The key driving requirements for MARTAS are the following: (i) the approach should provide guarantees for the adaptation goals with sufficient confidence (sufficient is defined by the stakeholders), and (ii) the approach should make adaptation decisions efficiently, paving the way to apply it to system settings at an increasing scale.

The central idea of MARTAS is to equip a MAPE loop with a separate runtime quality model for each adaptation goal. Each quality model takes care of one adaptation concern of the system (e.g. a stochastic automaton that models the packet loss of a network). Quality models can capture different uncertainties that are represented as model parameters (e.g. interference of network links or the traffic generated by motes). These parameters are monitored at runtime to update the runtime models. The adaptation goals are modeled as a set of rules that are used to select configurations that comply with the required adaptation goals (e.g., packet loss $< 10\%$, minimize energy consumption). To make an adaptation decision, the MAPE loop estimates the qualities for the different configurations that are considered for adaptation (i.e., the adaptation options).

To that end, the MAPE loop uses runtime statistical model checking (RSMC). RSMC combines runtime simulation with statistical techniques to estimate the qualities of each adaptation option with a required level of confidence. By combining the estimated qualities per adaptation option with the adaptation goals of the system, the MAPE loop can then select the best configuration to realize the adaptation goals. RSMC offers

an efficient verification approach compared to exhaustive approaches; it also allows balancing the confidence of the verified quality properties with the time and resources that are needed to compute them. MARTAS' modularity (separate definition of quality models and adaptation goals) also provides flexibility, paving the way for on-the-fly updating/changing models and goals.

Figure 2 shows a blueprint architecture of MARTAS.

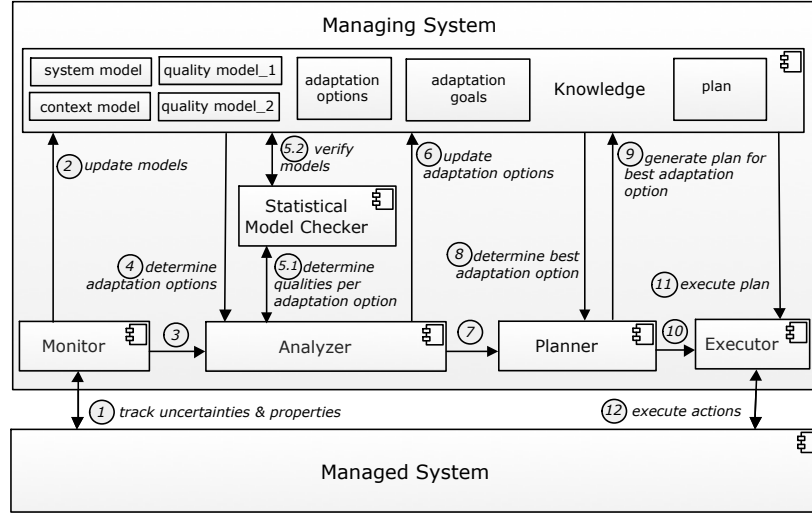


Fig. 2. Blueprint architecture of MARTAS

The approach works as follows:

1. The monitor tracks uncertainties and relevant properties of the managed system and the environment in which the system is deployment;
2. The collected data is used to update the corresponding runtime models;
3. The monitor triggers the analyser;
4. The analyser reads the adaptation options from the knowledge repository. The set of adaptation options is determined by the different configurations that can be selected to adapt the current configuration. An adaptation option has a placeholder for each quality property that is subject to adaptation;
5. For each adaptation option, the analyser estimates the expected qualities that are subject to adaptation. To that end, the analyser employs the statistical model checker that simulates the corresponding quality model with the settings of the adaptation option using a verification query in order to determine the expected quality with sufficient confidence (details are presented below).
6. The analyser updates the adaptation options with the verification results, i.e., it adds the estimated qualities to the placeholders for each adaptation option; it then determines whether the current configuration is able to achieve the adaptation goals;
7. If the current configuration is not able to achieve the adaptation goals the analyser triggers the planner;

8. The planner ranks the adaptation options by applying the adaptation goals to the different adaptation options and determines the best configuration;
9. The planner generates a plan for the best adaptation option;
10. The planner triggers the executor;
11. The executor executes the plan;
12. That is, the executor executes the adaptation actions to adapt the managed system.

In step 5, MARTAS employs RSMC. The central idea of RSMC is to check the probability $p \in [0, 1]$ that a model M satisfies a property φ , i.e., to check $P_M(\varphi) \geq p$ by performing a series of simulations. RSMC applies statistical techniques on the simulation results to decide whether the system satisfies the property with some degree of confidence. To verify a quality property it has to be formulated as a verification query. We use two types of queries: probability estimation ($p = Pr[bound](\varphi)$) and simulation ($simulate\ N[\leq\ bound]\{E1, \dots, Ek\}$). For a probability estimation query the statistical model checker applies statistical techniques to compute the number of runs needed to produce a probability estimation p for expression φ of the quality model with an approximation interval $[p - \epsilon, p + \epsilon]$ and confidence $(1 - \alpha)$ for a given time *bound*. The values of ϵ and α that determine the accuracy of the results can be set for each query. For a simulation query, the value of N determines the number of simulations the model checker will apply in time *bound* to return values for state expressions $E1, \dots, Ek$ of the quality model. For this type of query, the designer has to determine how many runs are needed to obtain a required accuracy. In our current research, we use the relative standard error of the mean (RSEM) as a measure to determine the accuracy of the simulation queries. The standard error of the mean (SEM) quantifies how precisely a simulation result represents the true mean of the population (and is thus expressed in units of the data). RSEM is the SEM divided by the sample mean and is expressed as a percentage. E.g., an RSEM of 5% represents an accuracy with a SEM of plus/minus 0.5 for a mean value of 10. In our current research, we rely on offline experiments only to compute the number of simulations required for a particular accuracy.

Note that the different adaptation goals applied in step 8 may not be completely independent, e.g., optimizing one of the goals may affect some other goals. In such cases, the order in which the goals are applied to the adaptation options may provide a means to determine the priority of goals.

4.2 Applying MARTAS to the IoT Application

We now show how we instantiated MARTAS to realise the adaptive IoT application. We start with an overview of the concrete architecture. Then we illustrate the runtime quality models that we used to estimate energy consumption together with the verification query. The evaluation and lessons learned are presented in the next sections.

To solve the problem of optimising and reconfiguring the IoT network, VersaSense and imec-DistriNet applied the innovative architecture-adaptive solution to the case. Figure 3 gives an overview of the layered architecture of the approach.

The bottom layer consists of the managed system with the network of motes and the gateway. The middle layer comprises a client that runs on a dedicated machine. This client offers an interface to the IoT network via a probe and an effector. The probe can be used to monitor the IoT network (status of motes and links, data about the packet loss,

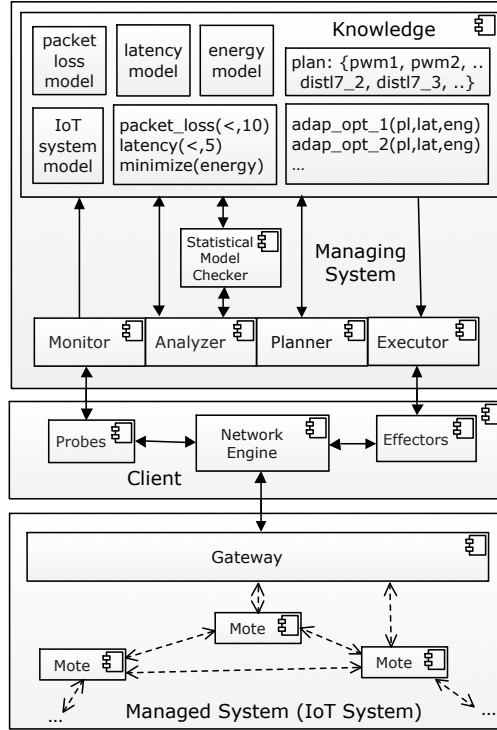


Fig. 3. MARTAS solution for the IoT application

the energy consumption, latency of the network, etc.) and the effector adapts the mote settings (power settings of the motes, distribution of packets sent to parents, etc.). The network engine collects data of the network in a repository and performs analyses on the data. In manual mode, an operator can access the IoT network via the client to track its status and perform reconfigurations manually. Reconfigurations include changing the power settings per communication link and changing the routing of packets by adapting the distribution of packets sent to parents. In the architecture-based adaptive solution, the top layer is added to the system that automatically adapts the configuration such that the adaptation goals of the IoT network are guaranteed.

The monitor uses the probe to track the traffic load and network interferences as well as the quality properties of interest. This data is used to update a set of models in the knowledge repository, including a model of the IoT system with the relevant aspects of the environment, and a set of quality models, one for each adaptation goal. As an example, Figure 4 shows the energy model that is specified as a set of timed automata.

The energy model is used by the statistical model checker to estimate the expected energy consumption per adaptation option (recall that the set of adaptation options is determined by the range of power settings of the motes for each outgoing link and the distribution of packets sent to parents). To that end it uses the following query:

simulate 1[<= 30] Gateway.avgEnergyConsumption

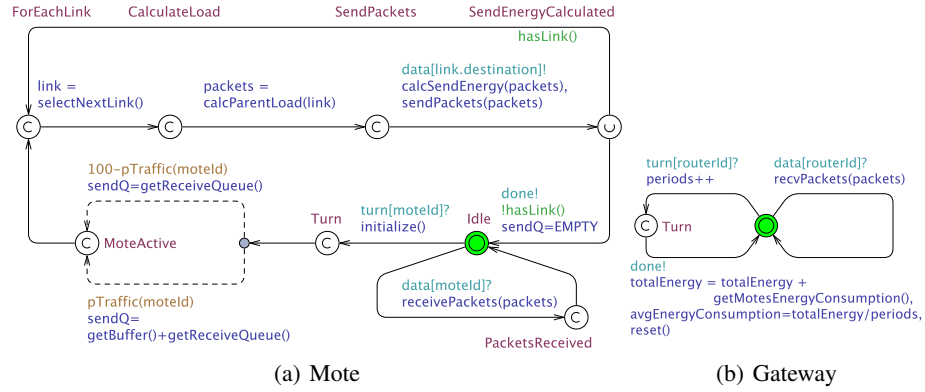


Fig. 4. Quality model used in the IoT application to predict energy consumption

This query performs 30 simulations to compute the expected average energy consumed for the next cycle. This number of simulations provides an RSEM of 0.5%, which was determined using offline experiments.

When the query is invoked, the model parameters for the adaptation option that is verified are set using the data collected by the monitor; e.g., the probability that each mote will generate traffic is set using the $pTraffic(moteID)$ parameter, for motes with multiple parents, the distributions of packets sent to parents are set, etc. The model is then simulated to determine its expected energy consumption. The automaton on the left represents a mote. The automaton is evaluated for each mote in the network in the order they communicate packets (time synchronised). When a mote gets its turn ($turn[moteID]$), the probability that it will send packets is determined based on its recently observed traffic load ($pTraffic(moteID)$). The mote then sends the packets in its queue to its parents one by one ($sendPackets(packets)$), i.e., the packets it received from children and the locally generated packets. As soon as the queue is empty ($sendQ=EMPTY$) the mote returns to the idle state. An idle mote can receive packets at any time ($receivePackets(packets)$). The automaton on the right shows the model for the gateway. When the gateway gets its turn, it computes the average energy consumption that was required to communicate packets in the cycle ($avgEnergyConsumption$).

When the different quality estimates are determined for all the adaptation options, the analyser updates the adaptation options, i.e., it adds the values for packet loss (pl), latency (lat), and energy consumption (eng) to the placeholders of the adaptation options ($adap_opt_i(pl,lat,eng)$, see Figure 4). The planner is then triggered to plan an adaptation if required. The planner starts by selecting the best adaptation option based on the quality properties determined during analysis. The planner then checks whether this option is: (i) in use, implying that no adaptation is required, (ii) no valid configuration is found, in this case a failsafe strategy is applied (i.e., the network is reconfigured to a default setting), (iii) a better option is found that can achieve the adaptation goals. If adaptation is required, the planner creates a plan consisting of steps, where each step either adapts the power setting of a mote for a link (e.g., $pwm1$ sets the power setting of mote 1) or it adapts the distribution of packets sent over a link to a parent of a mote

(e.g., *distl7.2*, sets the percentage of packets send by mote 7 over the link to mote 2). As soon as the plan is ready, the executor enacts the adaptation steps via the effectors.

Central to the novelty of MARTAS are two concepts that work in tandem: (i) a distinct runtime quality model for each adaptation goal, and (ii) the use of statistical model checking at runtime that performs a series of simulations and uses statistical techniques to estimate the qualities of the different adaptation options using the quality models. Compared to exhaustive model checking, statistical model checking is very efficient in terms of verification time and required resources. The tradeoff is that the results are not exact, but subject to a level of accuracy and confidence. The engineer can set this level, but higher confidence requires more time and resources.

5 Evaluation

We now evaluate MARTAS using the IoT application. We compare the novel approach with a conservative approach that is typically used in practice. Then we compare MARTAS with a state-of-the-art adaptation approach that uses exhaustive verification.

5.1 Comparison with Conservative Approach

We used the IoT application with 15 motes that is described in Section 3.2 (see also Figure 1) to compare MARTAS with a conservative approach that uses over-provisioning to deal with uncertainties (maximum power setting and all packets are sent to all parents). Each mote consists of: (1) a Raspberry Pi that is responsible for sensing, local processing, and network management operations, and (2) a LoRa module⁵ that deals with the radio communication. The gateway runs on a regular server machine.

As explained in Section 3.2, to adapt the IoT system, the power settings of the motes can be set from 0 to 15 (min to max power), and the distribution of packets sent over the links to multiple parents can be set in steps of 20%. This resulted in an adaptation space of in total 256 possible configurations.

We evaluated the packet loss, latency, and energy consumption of the IoT network for both approaches over a period of 24 hours. The IoT system was configured with a cycle time of 570 seconds, i.e., 9.5 minutes, hence a period of 24 hours consists of 153 cycles. Each cycle comprises 285 slots of 2 seconds. For each link, 40 slots are allocated for communication between the motes. During the first 8 minutes of the cycle the motes can communicate packets upstream to the gateway. The next half second is slack time and during the remaining 1 minute the gateway can communicate adaptation packets downstream to the motes. Each mote can generate up to 10 packets per cycle, depending on the type of sensor deployed and the conditions in the environment. The size of the queue is set to 60. Packets from children that arrive when the queue is full are discarded. The values for SNR are based on the actual conditions of the wireless communication. For the evaluation of MARTAS we used verification queries with an approximation interval of 1% and confidence of 90%, and simulations queries with a relative standard error of the mean of 0.5%. Figure 5 shows the test results.

⁵ <http://ww1.microchip.com/downloads/en/DeviceDoc/50002346C.pdf>

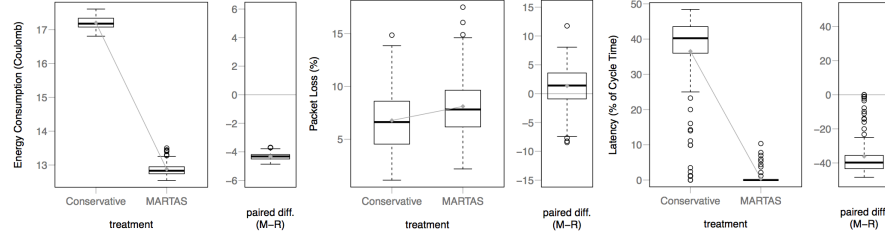


Fig. 5. Test results for conservative approach versus MARTAS

The boxplots show that the average energy consumption of MARTAS is significantly better compared to the conservative approach (p value < 0.000), and the same applies for latency ($p < 0.000$). For the packet loss, both approaches have similar results and realize the adaptation goal (mean of paired differences is 1.4%). Finally, the mean time required to realise adaptation is $45.7s$ $[44.5 \dots 48.5]$, which is only a fraction of the available adaptation time (8 minutes) for this setting. In conclusion, the test results demonstrate a substantial added value of applying self-adaptation, compared to the conservative approach that is typically used in practice.

5.2 Comparison with a State-of-the-Art Adaptation Approach

To compare MARTAS with runtime quantitative verification – RQV [3], a state-of-the-art adaptation approach that uses exhaustive verification, we used a simulation of the same IoT application setup used in the previous experiment⁶. We compare the packet loss and energy consumption of the IoT network, and the adaptation time for both approaches over a period of 12 hours, corresponding to 76 cycles. For the interference of links and the packets generated by the motes of the simulated system, we used profiles based on data that we collected from the physical deployment over a period of one week. For RQV, we translated the automata model for packet loss to a Discrete Time Markov Chain, and the model for energy consumption to a Markov Decision Process model. For runtime verification we used the PRISM model checker [17] with the default settings. Figure 6 shows the test results.

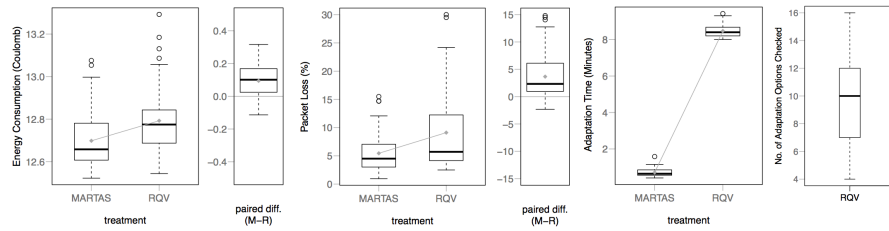


Fig. 6. Test results for MARTAS versus RQV

⁶ The simulator: <https://people.cs.kuleuven.be/danny.weyns/software/DeltaIoT/>

The boxplots show that the average energy consumption of MARTAS is slightly better compared to RQV (mean of 12.70 for MARTAS versus 12.79 for RQV, i.e., an energy saving of 4%). MARTAS achieves substantially better results for packet loss (mean of 5.45% versus 9.11%). The results for the adaptation time (that is primarily used for verification of adaptation options) show that MARTAS only uses a fraction of the available 8 minutes to select an adaptation option (43s [30 . . . 46]). RQV on the other hand consumes all the available time to select an adaptation option (508s (8m 28s) [787s . . . 529s]⁷). Moreover, the boxplot most right of Figure 6 reveals that within the available time of 8 minutes RQV was able to verify only a fraction of the available adaptation options (9 [6 . . . 12]). Testing the scalability of MARTAS in simulation (by increasing the number of motes and their connectivity) showed that the approach scales up to similar types of networks with 25 motes. In conclusion, MARTAS achieves better results compared to a state-of-the-art adaptation approach for all quality requirements, while it requires only a fraction of the available adaptation time. On the other hand, the results confirm the limitations of a state-of-the-art adaptation approach based on exhaustive verification in terms of the time it requires to make adaptation decisions.

6 Related Work

Various leading ICT companies have invested significantly in the study and application of self-adaptation techniques [14], including initiatives such as IBM's Autonomic Computing, Sun's N1, HP's Adaptive Enterprise, and Microsoft's Dynamic Systems. These efforts have resulted in industrial applications, such as automated server management, cloud elasticity and automated data center management. These efforts are often based on control-based adaptation solutions. For architecture-based adaptation on the other hand, there is still little understanding about the validity and tradeoffs of incorporating it into real-world software-intensive systems [5].

A number of recent R&D efforts have explored the application of architecture-based adaptation in practical applications. Georgas and Taylor [12] present a domain-independent approach for building adaptive robotic systems and discuss two case studies. Asadollahi et al. [1] apply the StarMX framework to self-manage an internet commerce environment that simulates the activities of a business oriented transactional web server. Happe et al. [13] argue for hierarchically structured models to manage self-healing and self-adaptation and discuss different viewpoints on this in the context of a robotic case. Camara et al. [5] apply architecture-based adaptation to an industrial middleware to monitor and manage highly populated networks of devices. Van Der Donckt et al. [26] investigated a novel cost-benefit adaptation schema for different QoS requirements and evaluated this approach on a real world IoT deployment. Recently, da Silva et al. [25] apply architecture-based techniques to role-based access control for business processes. While these related efforts apply architecture-based adaptation to practical applications, the only related effort that targets guarantees for the compliance of the systems with the adaptation goals is [25]. Overall, this paper contributes to existing efforts by applying a novel architecture-based adaptation approach in a different but important emerging domain: automation of the management of IoT networks.

⁷ RQV could complete the ongoing verifications that were started within 8 minutes.

7 Experiences and Lessons Learned

To conclude we report the results of a semi-structured interview we performed with two members of the technical staff of VersaSense. The aim was to get further insights in the experiences of the company with the self-adaptive solution. The interview was structured around six key questions. We summarise the main outcome of the interview.

Q1: What do you consider the most important benefits of the self-adaptive solution? Business and economical benefits, in particular: (i) removing manual interventions reduces costs to manage IoT deployments; (ii) being able to handle changes whenever they occur faster, (iii) achieving more optimal configurations in which the self-adaptive system is in the driver seat, avoiding conservative and sub-optimal configurations with as a result longer system lifetime (energy) and better customer experience; (iv) being able to offer 24/7 service to customers, without interruption.

Q2: What are potential risks of the self-adaptive solution and how could these risks be mitigated? Replacing experienced human interventions by self-adaptation requires trust and faith in the outcome the automated decision making. For IoT applications that are subject to very frequent changes, the overhead cost of self-adaptation may be an issue. E.g., when a system is optimized for a particular range of situations, and the operational context frequently changes, the system has to continuously adapt itself to accommodate new changes. A traditionally over-provisioned manually configured system might be able to accommodate more situations without the overhead of having to adapt itself at every change. A mitigation strategy could be finding a good balance between (1) a super-optimized self-adaptation policy for a particular situation, and (2) a less optimized policy that can accommodate more situations. Furthermore, good system monitoring, in combination with predicting patterns/trends of potential changes beforehand, might allow to determine a more optimal self-adaptation.

Q3: What are the most significant experiences gained from deploying and trialing the self-adaptive approach? The benefits of adaptation were already visible for small IoT deployments. The risks for the company are limited as the approach can be added as an add-on module so it can be tested in a controlled proof-of-concept setup. It will be particularly interesting to investigate now how the adaptive approach will scale towards bigger deployments with different application features and quality requirements.

Q4: Why do you think the adaptation approach presented in the paper is innovative or valuable for others. MARTAS is one of the first solutions that applies self-adaptation in the context of the new domain of IoT. It is one of the first solutions that actually applies automated management in the context of an existing IoT deployment.

Q5: Do you see a broader applicability of the proposed solution for your business? As IoT deployments and applications continue to grow in scale (i.e. number of assets being monitored and controlled by IoT devices), heterogeneity (types of devices and technologies involved), and complexity (i.e. application features and quality requirements), manual maintenance will require more and more efforts and become increasingly error-prone. We will investigate how the proposed solution can translate to other types of existing IoT applications, and how it can be integrated to the rest of the elements inside the infrastructure spanning from IoT devices towards cloud. Applying the proposed ap-

proach to automatically ensure optimal quality levels while keeping the entire system operating flawlessly, will certainly be a game-changer in this field.

Q6: What are your main lessons learned from this R & D effort? This interesting experiment showed how cutting-edge R & D efforts can be integrated with existing products and solutions. This may lead to improved products and additional competitive services that can be offered to customers. Solving the the problem of automating the management of IoT deployments required the combined expertise from researchers and engineers from different areas: software architectures, adaptation, IoT system engineering.

8 Conclusions

Architecture-based adaptation is a well-studied approach to mitigate uncertainties of software-intensive systems that are difficult to anticipate before deployment. The interference of the network links in an IoT system or the traffic generated by the motes are concrete examples of uncertainties that VersaSense faces that drove the research presented in this paper. However, existing adaptation approaches to handle uncertainties and provide guarantees for the required adaptation goals apply exhaustive verification techniques at runtime, which often limits their applicability in practice. To tackle this problem, we presented MARTAS, an innovative architecture-based adaptation approach to solve VersaSense problem: automating the management of Internet-of-Things (IoT). This novel approach equips an IoT application with a feedback loop that employs runtime models and statistical techniques to reason about the system and to adapt it to ensure the required goals. We applied MARTAS to a building security monitoring system, which is a representative case for a class of IoT applications deployed by VersaSense, and we demonstrated how MARTAS outperforms the current manual approach and a state-of-the-art approach for different quality goals. The main benefits of automating the management of IoT deployments are reduced costs, handle changes whenever they occur faster, achieve more optimal configurations, longer system lifetimes and better customer experience. The main risks are the need for trust in the automated decision making and potentially substantial overhead costs in settings that are subject to very frequent changes. These risks could be mitigated by balancing the quality optimality adaptation can achieve with the range of situation the approach can accommodate. This R & D effort demonstrated that solving a self-adaptation problem in practice required the combined expertise from researchers and engineers.

References

1. Asadollahi, R., et al.: Starmx: A framework for developing self-managing java-based systems. In: Software Engineering for Adaptive and Self-Managing Systems. IEEE (2009)
2. Brun, Y., et al.: Software Engineering for Self-Adaptive Systems. chap. Engineering Self-Adaptive Systems Through Feedback Loops. Springer-Verlag (2009)
3. Calinescu, R., et al.: Dynamic QoS Management and Optimization in Service-Based Systems. IEEE Transactions on Software Engineering **37**(3) (2011)
4. Camara, J., et al.: Assurances for Self-Adaptive Systems: Principles, Models, and Techniques. Lecture Notes in Computer Science Vol. 7740, Springer (2013)

5. Cámara, J., et al.: Evolving an adaptive industrial software system to use architecture-based self-adaptation. In: International Symposium on Software Engineering for Adaptive and Self-Managing Systems. IEEE Press (2013)
6. Cheng, R., et al.: Software Engineering for Self-Adaptive Systems: A Research Roadmap. Lecture Notes in Computer Science Volume 5525, Springer (2009)
7. Di Marzo Serugendo, G., Gleizes, M.P., Karageorgos, A.: Self-organization in multi-agent systems. *Knowledge Engineering Review* **20**(2) (2005)
8. Dobson, S., et al.: A survey of autonomic communications. *ACM Transactions on Autonomous and Adaptive Systems* **1** (2006)
9. Dustdar, S., et al.: Principles of elastic processes. *IEEE Internet Computing* **15**(5) (2011)
10. Galante, G., de Bona, L.: A survey on cloud computing elasticity. In: International Conference on Utility and Cloud Computing. IEEE Computer Society (2012)
11. Garlan, D., et al.: Rainbow: Architecture-Based Self-Adaptation with Reusable Infrastructure. *Computer* **37**(10) (2004)
12. Georgas, J., Taylor, R.: Policy-Based Architectural Adaptation Management: Robotics Domain Case Studies. Springer, Lecture Notes in Computer Science vol. 5525 (2009)
13. Happe, J., Koziolok, H., Bellur, U., et al.: The role of models in self-adaptive and self-healing systems. In: Self-Healing and Self-Adaptive Systems, Dagstuhl Report (2009)
14. Kephart, J.O., Chess, D.M.: The vision of autonomic computing. *Computer* **36**(1) (2003)
15. Kounev, S., et al.: Self-Aware Computing Systems. Springer Verlag (2017)
16. Kramer, J., Magee, J.: Self-Managed Systems: An Architectural Challenge. In: Future of Software Engineering. IEEE Computer Society (2007)
17. Kwiatkowska, M., Norman, G., Parker, D.: PRISM 4.0: Verification of probabilistic real-time systems. In: 23rd International Conference on Computer Aided Verification (2011)
18. de Lemos, R., et al.: Software Engineering for Self-Adaptive Systems: A Second Research Roadmap. Lecture Notes in Computer Science Volume 7475, Springer (2013)
19. de Lemos, R., et al.: Software Engineering for Self-Adaptive Systems: Research Challenges in the Provision of Assurances. Lecture Notes in Computer Science Vol. 9640 (2018)
20. Matthys, N., et al.: μ pnp-mesh: The plug-and-play mesh network for the internet of things. In: IEEE 2nd World Forum on Internet of Things (2015)
21. Oreizy, P., Medvidovic, N., Taylor, R.N.: Architecture-based runtime software evolution. In: International Conference on Software Engineering. IEEE Computer Society (1998)
22. Redwine, S., Riddle, W.: Software Technology Maturation. In: International Conference on Software Engineering. IEEE Computer Society Press (1985)
23. Salehie, M., Tahvildari, L.: Self-adaptive Software: Landscape and Research Challenges. *Transactions on Autonomous and Adaptive Systems* **4**, 14:1–14:42 (2009)
24. Shevtsov, S., et al.: Control-theoretical software adaptation: A systematic literature review. *IEEE Transactions on Software Engineering* **PP**(99) (2017)
25. da Silva, C.E., et al.: Self-adaptive role-based access control for business processes. In: Software Engineering for Adaptive and Self-Managing Systems. IEEE Press (2017)
26. Van Der Donckt, J., et al.: Cost-benefit analysis at runtime for self-adaptive systems applied to an iot application. In: Evaluation of Novel Approaches to Software Engineering (2018)
27. Weyns, D.: Software Engineering of Self-Adaptive Systems: An Organised Tour and Future Challenges. Chapter in Handbook of Software Engineering (2018, Springer), (forthcoming; <https://people.cs.kuleuven.be/danny.weyns/papers/2017HSE.pdf>)
28. Weyns, D., Iftikhar, U., Söderlund, J.: Do External Feedback Loops Improve the Design of Self-Adaptive Systems? A Controlled Experiment. In: International Symposium on Software Engineering of Self-Managing and Adaptive Systems. SEAMS '13 (2013)
29. Weyns, D., Malek, S., Andersson, J.: FORMS: Unifying Reference Model for Formal Specification of Distributed Self-adaptive Systems. *ACM TAAS* **7**(1), 8:1–8:61 (2012)
30. Weyns, D., et al.: Perpetual Assurances in Self-Adaptive Systems. In: Software Engineering for Self-Adaptive Systems, Lecture Notes in Computer Science, vol. 9640. Springer (2018)